

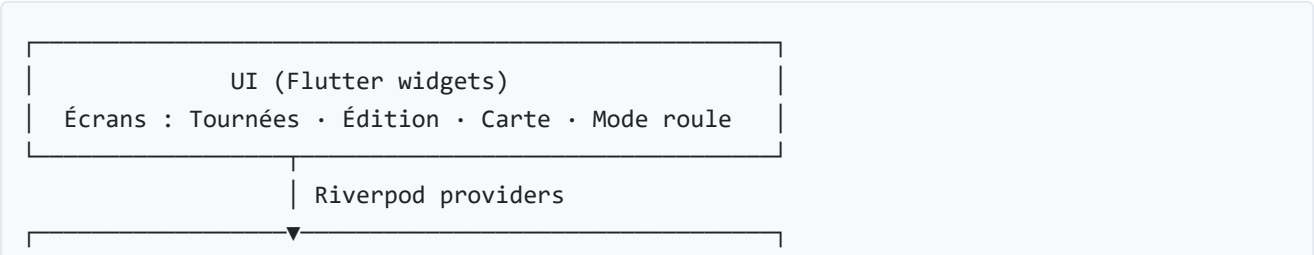
Plan détaillé — opti_route

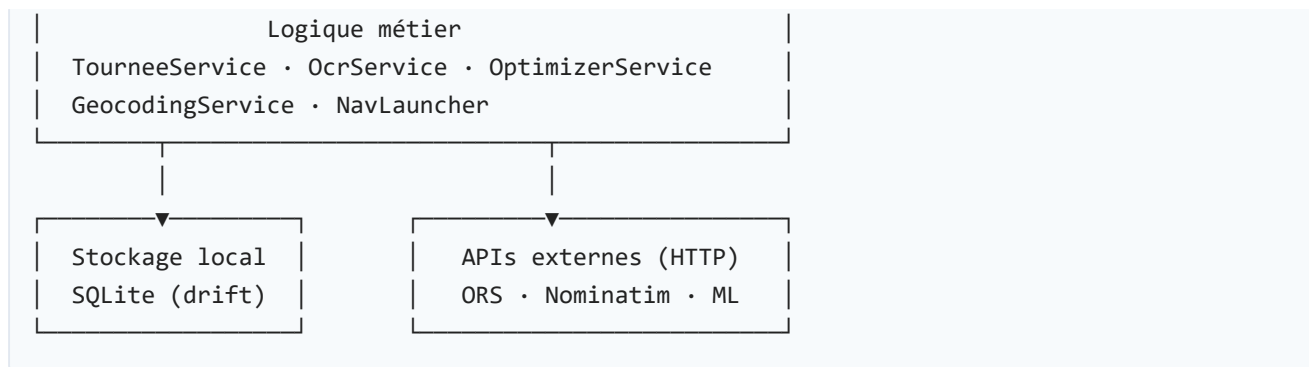
Application mobile Flutter (iOS + Android) pour l'optimisation de tournées de livraison multi-points, avec OCR d'adresses, contraintes par client, et navigation. **100% gratuit, sans carte de crédit.**

1. Stack technique

Couche	Outil	Pourquoi
Framework	Flutter 3.x (Dart)	Un seul code pour iOS + Android. Bonne perf. Excellents plugins de carte/OCR.
Carte	flutter_map + tuiles OpenStreetMap	Gratuit à vie, pas de clé API requise pour usage personnel raisonnable.
Optimisation tournée	OpenRouteService API (plan free)	500 optimisations/jour gratuites. Utilise VROOM, le meilleur solveur open-source. Pas de carte de crédit.
Itinéraires & temps	OpenRouteService API (plan free)	2000 directions/jour gratuites.
Géocodage (adresse → coordonnées)	Nominatim (OSM, public) + ORS en backup	Gratuit, public. Limite : 1 req/seconde — on bufferise.
Trafic temps réel (Phase 2)	TomTom Traffic free tier	2500 req/jour gratuit, sans carte de crédit.
OCR (caméra → texte)	google_mlkit_text_recognition	Tourne sur l'appareil , gratuit, pas de réseau requis.
Base de données locale	drift (au-dessus de SQLite)	Type-safe, idéal pour un débutant qui veut comprendre.
Gestion d'état	Riverpod	Recommandé en 2026, plus simple que Bloc, scalable.
Lancement nav externe	url_launcher + URL schemes Google Maps / Waze	Utilise la nav système (gratuite, déjà installée chez toi).

2. Architecture (vue d'ensemble)





Principe : tout le travail se fait localement (DB, OCR, UI). Les APIs externes ne sont appelées que pour : géocoder une nouvelle adresse, optimiser une tournée. Le reste fonctionne **hors ligne**.

3. Modèle de données

Table `tournees`

Colonne	Type	Description
<code>id</code>	int (PK)	
<code>nom</code>	text	Ex: « Tournée Mardi 12/05 »
<code>date</code>	date	Date prévue de la tournée
<code>point_depart_lat</code>	real	Adresse de départ (dépôt/maison)
<code>point_depart_lng</code>	real	
<code>point_depart_label</code>	text	
<code>vehicule_capacite_colis</code>	int	Pour optim multi-tournées plus tard
<code>statut</code>	text	<code>brouillon</code> / <code>optimisee</code> / <code>en_cours</code> / <code>terminee</code>
<code>cree_le</code>	datetime	

Table `stops`

Colonne	Type	Description
<code>id</code>	int (PK)	
<code>tournee_id</code>	int (FK)	
<code>adresse_brute</code>	text	Tel que saisi/scanné
<code>adresse_normalisee</code>	text	Retournée par le géocodeur
<code>lat</code>	real	
<code>lng</code>	real	

Colonne	Type	Description
<code>nb_colis</code>	int (default 1)	
<code>priorite</code>	text	<code>obligatoire_premier</code> / <code>obligatoire_dernier</code> / <code>flexible</code> / <code>eviter_si_possible</code>
<code>fenetre_debut</code>	time?	Ex: « pas avant 9h »
<code>fenetre_fin</code>	time?	Ex: « avant midi »
<code>duree_arret_min</code>	int (default 3)	Temps estimé sur place
<code>notes</code>	text?	Ex: « code 1234B », « porte côté garage »
<code>nom_client</code>	text?	
<code>statut_livraison</code>	text	<code>a_livrer</code> / <code>livre</code> / <code>echec</code> / <code>reporte</code>
<code>ordre_optimise</code>	int?	Position 1..N après optimisation
<code>cree_le</code>	datetime	

Table `parametres`

Colonne	Type	Description
<code>cle</code>	text (PK)	Ex: <code>ors_api_key</code> , <code>tomtom_api_key</code>
<code>valeur</code>	text	

4. Écrans (parcours utilisateur)

Écran 1 — Liste des tournées (accueil)

- Liste des tournées avec date, statut, nb d'arrêts.
- Bouton flottant « + Nouvelle tournée ».
- Tap sur une tournée → écran 2 (détail).

Écran 2 — Détail/édition d'une tournée

- Onglet **Arrêts** : liste des stops, drag pour réordonner manuellement, swipe pour supprimer.
- Onglet **Carte** : aperçu sur carte de tous les arrêts.
- Onglet **Réglages** : nom, date, point de départ, capacité.
- Trois boutons d'ajout d'arrêt : 1.  Saisie manuelle → écran 3a 2.  Scanner avec caméra → écran 3b 3.  Importer une image → écran 3c
- Bouton «  Optimiser la tournée » (appelle ORS, met à jour `ordre_optimise`).
- Bouton «  Démarrer la tournée » (passe en mode roule, écran 5).

Écran 3a — Saisie manuelle d'un arrêt

- Champ adresse (autocomplete via Nominatim).
- Nom client, nb colis, priorité (radio), fenêtre horaire (optionnel), notes.
- Bouton « Enregistrer ».

Écran 3b — Scanner caméra

- Vue caméra plein écran (`camera` package).
- Bouton « Capturer ».
- Après capture : OCR avec ML Kit → texte extrait.
- Écran de confirmation : on surligne ce qui ressemble à une adresse, l'utilisateur valide/corrige.
- Géocodage → préremplissage écran 3a.

Écran 3c — Import image

- Sélection depuis galerie (`image_picker`).
- Même flux qu'écran 3b à partir de l'OCR.

Écran 4 — Vue carte + ordre optimisé

- Carte avec marqueurs numérotés 1..N.
- Tracé du trajet (polyline retournée par ORS).
- Liste latérale (ou bottom sheet) avec ordre.
- Estimation totale : durée, distance.

Écran 5 — Mode « en cours » (roule)

- Affiche le **stop courant** en grand (adresse, client, notes, nb colis).
- Bouton «  Naviguer » → lance Google Maps ou Waze externe.
- Bouton «  Livré ».
- Bouton «  Échec / Reporter ».
- Mini-carte avec progression.

Écran 6 — Paramètres

- Clés API (ORS, TomTom plus tard).
- Adresse de départ par défaut.
- Préférences (nav par défaut : Google Maps ou Waze).

5. Ordre des tâches (jalons Phase 1)

Chaque jalon = quelque chose qui marche et que tu peux tester.

#	Jalon	Estimation débutant
1	Setup : Flutter installé, projet créé, app vide qui démarre sur émulateur.	1 journée

#	Jalon	Estimation débutant
2	Base de données : drift configuré, tables créées, on peut insérer/lire en code.	1-2 jours
3	Liste tournées + créer/éditer : UI minimale, persistance OK.	2-3 jours
4	Saisie manuelle d'arrêts + édition complète des champs.	2-3 jours
5	Géocodage Nominatim : transformer une adresse en lat/lng.	1-2 jours
6	Affichage carte : <code>flutter_map</code> avec marqueurs des stops.	1-2 jours
7	Optimisation ORS : appel API, mise à jour <code>ordre_optimise</code> , tracé du trajet.	3-5 jours
8	OCR : caméra → texte → extraction adresse → préremplissage formulaire.	4-7 jours (le plus dur)
9	Mode roule : stop courant, lancement nav externe, marquer livré.	2-3 jours
10	Polish : icônes, empty states, validation, gestion d'erreurs réseau.	2-3 jours

Total Phase 1 estimé pour un débutant motivé : 6 à 10 semaines (à raison de quelques heures par jour).

6. Comptes & clés à créer (gratuit, sans CB)

Avant de coder le jalon 7 (optimisation), il te faudra :

1. **OpenRouteService** : créer un compte sur <https://openrouteservice.org/dev/#/signup> — pas de CB demandée, clé API gratuite immédiate.
2. **Nominatim** : aucun compte requis pour usage léger (<1 req/sec), juste un User-Agent identifiant ton app.
3. (Phase 2) **TomTom** : <https://developer.tomtom.com> — gratuit jusqu'à 2500 req/jour, sans CB.

7. Ce qui est volontairement hors scope Phase 1

- ❌ CarPlay / Android Auto (Phase 3, exige du code natif Swift/Kotlin)
- ❌ Trafic temps réel (Phase 2)
- ❌ Synchronisation cloud / multi-appareils
- ❌ Authentification, comptes utilisateurs
- ❌ Paiement, abonnements
- ❌ Mode hors-ligne complet pour les cartes (utilisable en ligne uniquement Phase 1 ; cartes offline = Phase 2)
- ❌ Voix / commande vocale

8. Risques & points d'attention

Risque	Atténuation
Quota ORS dépassé (500 optim/jour)	Largement suffisant pour un usage perso. On affichera le quota restant.
OCR peu fiable sur bordereaux froissés	Toujours montrer le texte extrait à l'utilisateur pour validation avant géocodage.
Géocodage rate une adresse	Fallback : permettre placement manuel sur carte par tap.
Nominatim rate limit	Buffer 1 sec entre requêtes, cache local des adresses déjà géocodées.
Tuiles OSM bannies si abus	Respecter la policy (User-Agent, pas de scraping massif). Pour usage perso : aucun souci.
Code natif iOS pour CarPlay	Phase 3 — faudra apprendre du Swift basique ou trouver un plugin maintenu.

9. Setup machine (à faire avant le jalon 1)

Sur ton Windows 11 :

1. **Flutter SDK** — <https://docs.flutter.dev/get-started/install/windows>
2. **Android Studio** (pour SDK Android et émulateur) — <https://developer.android.com/studio>
3. **VS Code** + extensions « Flutter » et « Dart » (alternative légère à Android Studio pour coder)
4. **Git** pour versionner ton code — <https://git-scm.com/download/win>
5. Vérification finale avec `flutter doctor` — doit afficher tous les ✓

Pour le test sur iPhone réel + plus tard CarPlay, il faudra un Mac. Sur Windows, on peut développer pour iOS mais on ne peut **pas** compiler/tester l'app iOS. Pour l'instant on se concentre sur Android (émulateur sur ton PC + ton téléphone Android si tu en as un).

10. Question ouverte

As-tu un téléphone Android pour tester l'app sur du vrai matériel ? Sinon l'émulateur Android Studio suffit pour la majorité du dev. Pour l'iPhone et CarPlay : il faudra un Mac (location à l'heure sur MacInCloud à ~1 \$/h, ou achat d'un Mac mini d'occasion plus tard).